# Assignment 1: How Runtime and Memory Scale in the Basic Triple-Loop Matrix Multiplication in C, Java and Python

Hugo González Fernández
Big Data · Bachelor's Degree in Data Science and Engineering
Universidad de Las Palmas de Gran Canaria
Academic year 2026–2027 · Individual Assignment 1

October 2026

**Abstract**

**Question.** How do the runtime and the memory use of the textbook $O(n^3)$ multiplication of $n \times n$ `float64` matrices grow with $n$ in C, Java and Python? **Method.** Three implementations of the same i–j–k triple loop read identical, seeded input files; only the multiplication kernel is timed (monotonic timers, warm-up and repeated samples, median and IQR); peak resident memory is recorded per process; every result is validated against NumPy with an a-priori error bound and the three languages produce bit-identical output. Budgets were 300 s per multiplication and 4 GiB. **Results.** Time grows as $\approx n^{3.2}$ for $n \leq 400$, and the cost per multiply–add increases for larger matrices (C 1.0–1.4 $\rightarrow$ 6.8 ns, Java 1.6 $\rightarrow$ 11.1 ns). Python is 49–75× slower than C and Java at $50 \leq n \leq 1000$. Java `double[n][n]` falls 2.2–4.3× behind C beyond $n \approx 1000$, and a supplementary flat-array variant removes most of the gap; C shows $\approx$ 1.8–1.9× spikes at $n = 512$ and 1024. Memory follows array storage (24, $\approx$ 22 and $\approx$ 121 bytes per $n^2$) plus a runtime baseline (1.8, 45.7, 9.8 MiB). The largest feasible sizes were 3000 (C), 2048 (Java) and 1250 (Python). **Scope.** One virtual machine, one compiler/JVM/interpreter version each, 2–10 repetitions: the conclusions are valid for these settings only and are not a general ranking of languages.

## 1 Introduction and objective

Dense matrix multiplication is the standard microbenchmark for studying how a programming language implementation turns the same algorithm into machine work. This study compares the *textbook triple-loop* algorithm, which performs $n^3$ multiply–add operations on $n \times n$ matrices, in three languages: C (ahead-of-time compiled), Java (JIT-compiled on the JVM) and Python (CPython, interpreted).

**Research question.** *How do the runtime and the memory use of the basic $O(n^3)$ multiplication of $n \times n$ `float64` matrices grow with $n$ in C, Java and Python, when the three programs use the same inputs, the same loop order and the same arithmetic on one documented machine?*

**Factor and outcomes.** The factor that changes is the matrix size $n$ (from 10 up to the largest size that fits the declared budgets); the outcomes are the time of one multiplication (kernel only) and the peak resident memory of the process.

**Hypotheses (predictions, stated before running the experiments).**

- H1 In every language the time grows approximately as $n^3$ (log–log slope $\approx 3$), but the constant factor differs.
- H2 The order of speed is C $\lesssim$ Java $\ll$ Python, with Python one to two orders of magnitude slower than the compiled languages, because of interpreter dispatch and boxed floating-point objects.
- H3 The cost per multiply–add is not constant: once the matrix $B$, which is read by columns, no longer fits in a cache level, the cost per operation increases.

H4 Memory is dominated by array storage: about $24\,n^2$ bytes for C and Java (3 matrices $\times 8$ bytes), a much larger $\approx 120\,n^2$ bytes for Python (each element is an 8-byte pointer plus a separately allocated float object), plus a fixed runtime baseline that is largest for the JVM.

**Scope.** Conclusions are limited to the implementations, compiler and runtime settings, inputs and the virtual machine described in Section 3.3. They are *not* a general ranking of programming languages. Language-specific optimisations (loop reordering, blocking, libraries) are deliberately excluded; they belong to the later assignments.

# 2 Background

**Algorithm.** For $C = AB$ with $A, B \in \mathbb{R}^{n\times n}$, $c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$. The direct evaluation needs $n^3$ multiplications and $n^3 - n^2$ additions, hence $\Theta(n^3)$ work [1]. All three programs use the loop order $i$–$j$–$k$ and accumulate each $c_{ij}$ in a scalar before storing it.

**Memory access pattern.** With row-major storage the inner loop walks $a_{ik}$ along a row (unit stride) but $b_{kj}$ down a column (stride $8n$ bytes). Once a column of $B$ touches more cache lines than the cache can keep for the next value of $j$, every access to $B$ becomes a cache miss; for stride $8n$ with $n$ a power of two, many addresses additionally map to the same cache sets [2, 3]. This is why the cost per multiply–add is expected to rise with $n$ even though the operation count is exactly cubic.

**Execution models.** C code is compiled to machine code before running. Java bytecode starts interpreted and is compiled by HotSpot's tiered JIT (C1, then C2), including on-stack replacement of hot loops; array accesses keep bounds checks unless the JIT can remove them. CPython executes bytecode in an interpreter: every `A[i][k]*B[k][j]` involves several bytecode dispatches, index operations and the allocation of a new `float` object.

**Floating-point error.** For a sum of $n$ products, any evaluation order gives $|\hat{c} - c| \le \gamma_n \sum_k |a_{ik} b_{kj}|$ with $\gamma_n = nu/(1-nu)$ and $u = 2^{-53}$ [4]. Because all entries here are non-negative, $\sum_k |a_{ik} b_{kj}| = c$, so each implementation has relative error at most $\gamma_n$, and two different evaluation orders differ by at most $2\gamma_n$ [5]. This bound is used as the acceptance tolerance (Section 3.9).

**Benchmarking practice.** The protocol follows common recommendations: report central value and spread over repeated runs, separate warm-up from steady state for JIT-compiled languages, state exactly what is timed and report the environment [6, 7].

# 3 Methodology

## 3.1 Implementations

- **C** (`src/c/matmul.c`): one contiguous row-major block of $n^2$ `double` per matrix; the kernel is a separate non-inlined function so repetitions cannot be merged by the compiler.
- **Java** (`src/java/MatMul.java`): `double[n][n]`, i.e. $n$ separately allocated row arrays; kernel in its own static method.
- **Python** (`src/python/matmul.py`): list of $n$ lists of $n$ boxed `float`; pure Python loops, *no* NumPy in the kernel.

The kernel is identical in structure in all three: `s = 0.0; for k: s += A[i][k]*B[k][j]; C[i][j] = s`, for $i$, $j$ in this order. The scalar accumulator differs slightly from a `C[i][j] += ...` form: it keeps the same operation order (so the results are bit-for-bit identical, see Section 3.9) and avoids language-specific aliasing effects in the comparison. Unavoidable differences are the storage layouts above and the runtime behaviour (JIT, interpreter, boxing). No library multiplication is used by any implementation.

## 3.2 Inputs

Matrices $A$ and $B$ for each $n$ are generated once with NumPy's `default_rng` (PCG64) seeded with $[20261012, n, w]$ ($w = 0$ for $A$, $w = 1$ for $B$), uniform in $[0, 1)$, and stored as raw little-endian binary64 files. All three programs read the *same files*, so the logical inputs are identical and no cross-language random generator is involved. SHA-256 digests of all files are stored in `data/manifest.json`.

## 3.3 Environment

All measurements were taken on one machine, sequentially, with no other workload running (the driver only polled the child process every 0.5 s). Table 1 lists the environment. The machine is a virtual machine (KVM) with 2 vCPUs; the frequency governor is not visible and the processes were not pinned to a CPU. Each kernel is single-threaded; the JVM may place its JIT and GC helper threads on the second vCPU. Thread counts used by numerical libraries are not relevant because no library is used in the kernels (NumPy is only used for generating inputs and checking results, *outside* the measurements) [8].

| Item | Value |
|---|---|
| CPU | Intel(R) Xeon(R) Processor @ 2.80GHz |
| Cores / logical CPUs | 2 physical (vCPU), 2 logical; 1 thread(s)/core |
| Caches | L1d 64 KiB (2 instances), L2 2 MiB (2 instances), L3 33 MiB (1 instance) |
| RAM | 8030 MiB, swap 0 MiB |
| Virtualisation | KVM (full) |
| OS / kernel | Ubuntu 24.04.5 LTS, Linux 6.18.44-fc-v114 |
| Python | CPython 3.13.16 (GCC 13.3.0) |
| Java | OpenJDK 21.0.12.1 (HotSpot 64-bit server VM); flags -XX:+UseSerialGC -Xms256m -Xmx4g (default tiered JIT: C1+C2) |
| C compiler | gcc (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0; flags -O2 -std=c11 -ffp-contract=off (no -march=native; no -ffast-math) |
| NumPy (reference only) | 2.5.3 |
| Threads in the kernel | 1 (single-threaded); no CPU pinning; frequency governor: not exposed (virtual machine) |

**Table 1:** Hardware and software environment.

## 3.4 Fixed settings

Fixed in every comparison: algorithm and loop order, numeric type (binary64), inputs, single thread, one process per configuration, monotonic timers, kernel-only timing. C is compiled with `gcc -O2 -std=c11 -ffp-contract=off` (no `-march=native`, no `-ffast-math`): this forbids fused multiply–add and re-association, so the arithmetic is the same as in Java and Python. Java runs on OpenJDK 21 with the default tiered JIT and `-XX:+UseSerialGC -Xms256m -Xmx4g`. Python is CPython 3.13 with the cyclic garbage collector disabled inside the timed region (the kernel allocates only floats, which are not GC-tracked, so this only removes a source of pauses). Different compiler options for C are explored in a *separate* supplementary experiment (Section 4.6).

## 3.5 What the timer includes

**Kernel only.** The timed region is the triple loop. Reading the inputs, allocating $C$, zero-initialising $C$ (so its pages are touched before timing), computing the checksum and dumping the result are *outside* the timer. In Python the row lists of $C$ are created outside the timer; the float objects that hold the results are created by the kernel and are therefore inside it. Timers: `clock_gettime(CLOCK_MONOTONIC)` (C), `System.nanoTime()` (Java), `time.perf_counter_ns()` (Python). The result of every multiplication is consumed: after each timed sample a checksum of $C$ (plain left-to-right sum in row-major order) is computed and recorded, and the final $C$ is written to disk and verified.

## 3.6 Repetition, warm-up and summary statistics

Each configuration (language, $n$) runs in its own process with a number of *warm-up* samples (excluded from the statistics but kept in the raw data) followed by *measured* samples. The numbers depend on the

estimated time of one multiplication $\hat{t}$: $\hat{t} \le 1\,\text{s}$: 3 warm-up, 10 measured; $\le 30\,\text{s}$: 1 warm-up, 5 measured; $\le 120\,\text{s}$: no warm-up, 3 measured; larger: no warm-up, 2 measured. Fewer repetitions for costly runs are a compromise forced by the budget (a single Python multiplication at the largest sizes takes minutes); the resulting spread is reported through min–max and IQR, and the number of repetitions is given for every configuration (Table 3). When $\hat{t} < 50\,\text{ms}$ a batch of $m = \lceil 0.1\,\text{s}/\hat{t}\,\rceil$ multiplications is timed and divided by $m$, and the reported value is the *time per multiplication*. Reported statistics are the **median** and the **inter-quartile range** (IQR, $Q_3 - Q_1$). The runs for different languages are interleaved by size (for each $n$: C, Java, Python) so that slow drift of the machine affects all languages alike.

## 3.7 Budgets and limits

Declared before the large runs: a single timed sample must complete within **300 s**; peak resident memory per run must not exceed **4 GiB**. A run that violates a limit is killed and recorded as a *timeout* or *memory failure*, never given an invented time. The driver does not attempt sizes whose predicted time exceeds three times the limit and stops enlarging $n$ for an implementation after its first failure. The *largest feasible* size (all repetitions completed, results validated) and the *largest attempted* size are reported separately (Table 7). Planning aid: three binary64 $n \times n$ matrices need $24n^2$ bytes (e.g. $\approx 0.4\,\text{GB}$ at $n = 4096$), so memory was not expected to bind before time did.

## 3.8 Memory metric

The primary metric is the **peak resident set size** of the whole process (`VmHWM` from `/proc/self/status`, read by each program just before it exits and before the result is dumped), so it includes the runtime itself (JVM, interpreter) and all matrices. The kernel's own accounting (`ru_maxrss` from `wait4`) is stored as a cross-check. RSS is not the same quantity as managed-heap use or as the arithmetic estimate of array storage, and it is not comparable with them directly; the model lines in Figure 3 are estimates of array storage added to the measured baseline of the smallest run. **Limitations:** RSS counts pages actually resident (not reserved), includes loader/library pages, and has granularity of one page; the Python process also needs transient buffers while loading rows.

## 3.9 Correctness

Before any benchmark, `scripts/test_correctness.py` runs all three programs on small cases whose answers are known (scalar, hand-computed $3 \times 3$, non-commutative $2 \times 2$, zero matrix, identity on both sides, all-ones, small-integer matrices; *exact* equality required, all intermediate values are integers below $2^{53}$, so there is no overflow or rounding), and on random matrices against NumPy's `float64` product (tolerance $2\gamma_n$ relative, derived in Section 2; absolute tolerance 0 because every reference entry is positive). It also requires the three languages to produce *bit-for-bit identical* results. During the benchmark every run dumps its final product and the driver checks it element-wise against NumPy with the same tolerance, plus the checksum against the reference sum (tolerance $\gamma_{n^2+2n}$). NumPy/BLAS serves only as a trusted reference and is not one of the implementations under study.

## 3.10 Supplementary experiments (exploratory, kept separate)

The following experiments were designed *after* looking at the main results, to test explanations for patterns seen there; they are therefore exploratory, they are reported separately, and they do not change the controlled comparison above. All use the same driver, inputs, budgets and validation.

1. **C optimisation level** (Section 4.6): the C program compiled with `-O0`, `-O1`, `-O2`, `-O3` and `-O3 -march=native` (always with `-ffp-contract=off`) at $n \in \{512, 1000, 1024\}$.
2. **Sizes around powers of two** (Section 4.6): $n \in \{496, 504, 512, 520\}$ and $\{1016, 1024, 1032\}$ for C, Java, and the variant below.
3. **Java with a flat array** (`src/java/MatMulFlat.java`): the same Java algorithm with each matrix stored as one row-major `double[n*n]` (the layout used by the C program), to test whether the jagged `double[n][n]` layout explains the Java–C gap. Sizes $n \in \{500, 1000, 1250, 1500, 2000\}$ and the power-of-two neighbourhoods. Correctness is checked by the driver on every run (same tolerance), not by the unit-test script.

4. **Between-process variability** (Section 4.6): C and Java at $n = 1024$ in six independent processes (the main run, the power-of-two run and four dedicated repeats).

# 4 Results

All values are medians of the measured samples; values in parentheses are IQRs. Times are *per multiplication*, kernel only (Section 3.5). The raw samples (including warm-up) are in `results/raw_measurements.csv`; per-run information is in `results/runs.csv`.

## 4.1 Correctness

All 52 checks of the pre-benchmark test script passed (Table 2); the maximum observed relative error against NumPy was $7.6 \times 10^{-16}$ ($n = 100$), far below the a-priori bound. In the benchmark, all **92 completed runs** (43 of the main campaign and 49 supplementary) passed the element-wise check and the checksum check (largest relative error $8.3 \times 10^{-15}$, i.e. below 10 % of the accepted bound; per-run values in `runs.csv`), and the final $C$ of C, Java and Python was **bit-for-bit identical** for every size where more than one language completed (SHA-256 of the dumped matrix, `results/summary_bitwise.csv`). The identical bits are expected because all three programs use the same operation order, binary64 arithmetic and no fused multiply–add.

| Test (all three languages) | Check | Max. rel./abs. error | Tolerance | Result |
|---|---|---|---|---|
| scalar 1x1 (3*4) | exact | 0 | 0 (exact) | pass |
| hand-computed 3x3 | exact | 0 | 0 (exact) | pass |
| 2x2 non-commutative | exact | 0 | 0 (exact) | pass |
| zero * random (n=7) | exact | 0 | 0 (exact) | pass |
| identity * B (n=16) | exact | 0 | 0 (exact) | pass |
| A * identity (n=16) | exact | 0 | 0 (exact) | pass |
| all-ones (n=32) -¿ every entry 32 | exact | 0 | 0 (exact) | pass |
| random integers 0..99 (n=64), exact | exact | 0 | 0 (exact) | pass |
| random [0,1) n=5 vs NumPy | tol | 1.8e-16 | 1.1e-15 | pass |
| random [0,1) n=17 vs NumPy | tol | 2.2e-16 | 3.8e-15 | pass |
| random [0,1) n=33 vs NumPy | tol | 5.6e-16 | 7.3e-15 | pass |
| random [0,1) n=64 vs NumPy | tol | 4.3e-16 | 1.4e-14 | pass |
| random [0,1) n=100 vs NumPy | tol | 7.6e-16 | 2.2e-14 | pass |

**Table 2:** Pre-benchmark correctness tests (each run in C, Java and Python; bit-for-bit identical across languages in every test).

## 4.2 Runtime versus matrix size

Table 3 and Figure 1 show the runtime. For small sizes the values come from batches (Section 3.5); at the largest sizes only 2–3 repetitions were affordable. At $n \geq 1500$ Python hit the 300 s limit and from $n = 3000$ Java did (Table 7).

| $n$ | C | Java | Python | reps (C/J/P) |
|---|---|---|---|---|
| 10 | 0.526 µs (0.0102 µs) | 0.949 µs (0.168 µs) | 57.5 µs (7.37 µs) | 10/10/10 |
| 50 | 95.7 µs (3.25 µs) | 119 µs (23.4 µs) | 7.13 ms (604 µs) | 10/10/10 |
| 100 | 816 µs (38.8 µs) | 990 µs (83.6 µs) | 50.1 ms (3.56 ms) | 10/10/10 |
| 200 | 8.39 ms (261 µs) | 9.12 ms (112 µs) | 487 ms (52.8 ms) | 10/10/10 |
| 256 | 19.8 ms (1.36 ms) | 19.9 ms (2.91 ms) | 1.07 s (91.4 ms) | 10/10/5 |
| 400 | 81.1 ms (10.4 ms) | 100 ms (7.84 ms) | 5.25 s (165 ms) | 10/10/5 |
| 500 | 154 ms (5.88 ms) | 198 ms (10.3 ms) | 11.4 s (93.9 ms) | 10/10/5 |
| 512 | 284 ms (39.7 ms) | 219 ms (23.6 ms) | 12.8 s (260 ms) | 10/10/5 |
| 800 | 700 ms (11.8 ms) | 831 ms (76.9 ms) | 46.3 s (920 ms) | 5/5/3 |
| 1000 | 1.33 s (30.6 ms) | 2.02 s (118 ms) | 98.9 s (1.79 s) | 5/5/3 |
| 1024 | 3.34 s (196 ms) | 3.21 s (94.8 ms) | 107 s (1.16 s) | 5/5/2 |
| 1250 | 4.33 s (1.55 s) | 10.2 s (201 ms) | 207 s (940 ms) | 5/5/2 |
| 1500 | 6.46 s (332 ms) | 28 s (321 ms) | *timeout* | 5/5/– |
| 2000 | 33.4 s (600 ms) | 86.9 s (494 ms) | *not attempted* | 5/3/– |
| 2048 | 44.2 s (2.89 s) | 95 s (541 ms) | *not attempted* | 3/2/– |
| 3000 | 185 s (981 ms) | *timeout* | *not attempted* | 2/–/– |

**Table 3:** Median (IQR) time of one multiplication, and number of measured repetitions per implementation. Italics: no time exists for that configuration (timeout at the 300 s per-sample limit, or not attempted because an earlier size already failed).

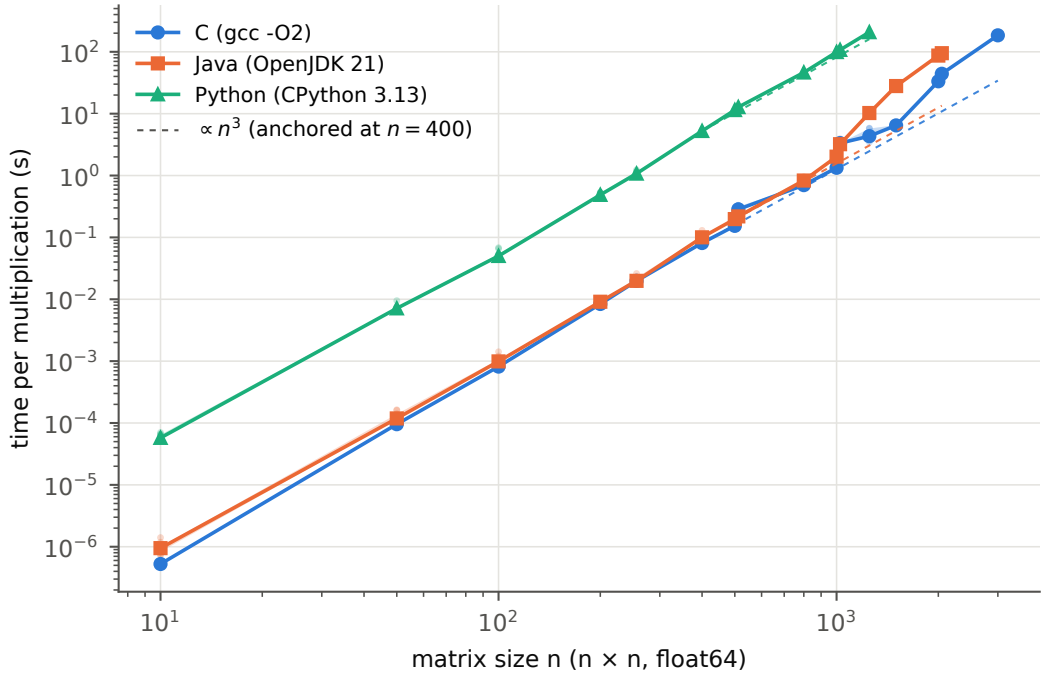



**Figure 1:** Median time per multiplication against $n$ (log–log). Bands: IQR; faint points: individual repetitions; dashed lines: $n^3$ growth anchored at $n = 400$. Python was run only up to $n = 1250$ and Java up to $n = 2048$ within the budget.

**Growth.** A least-squares fit of $\log t$ against $\log n$ gives exponents close to the cubic model only for small to medium sizes (Table 4): 3.17–3.27 for $50 \le n \le 400$ in all three languages. Over the larger windows the exponent is 3.29 for Python, 3.63 for C and 4.01 for Java, i.e. the cost per multiply–add is not constant. Figure 2 shows this directly: the time per multiply–add $t/n^3$ is almost flat for Python and for C/Java up to $n \approx 1000$, and then grows (C: 1.3 → 6.8 ns, Java: 2.0 → 11.1 ns). At $n = 512$ and $n = 1024$ C has isolated spikes (Section 4.6).

| Impl. | Sizes used ($n$) | Points | Exponent $k$ | 95 % CI |
|---|---|---|---|---|
| C | 50–400 | 5 | 3.27 | [3.16, 3.38] |
| C | 200–3000 | 13 | 3.63 | [3.37, 3.89] |
| Java | 50–400 | 5 | 3.22 | [3.04, 3.40] |
| Java | 200–2048 | 12 | 4.01 | [3.67, 4.36] |
| Python | 50–400 | 5 | 3.17 | [2.92, 3.43] |
| Python | 200–1250 | 9 | 3.29 | [3.19, 3.39] |

**Table 4:** Empirical exponent $k$ in $t \propto n^k$ (least squares on the medians, 95 % $t$-interval). The interval reflects the scatter of the medians around the line, not between-process variability (Section 4.6).

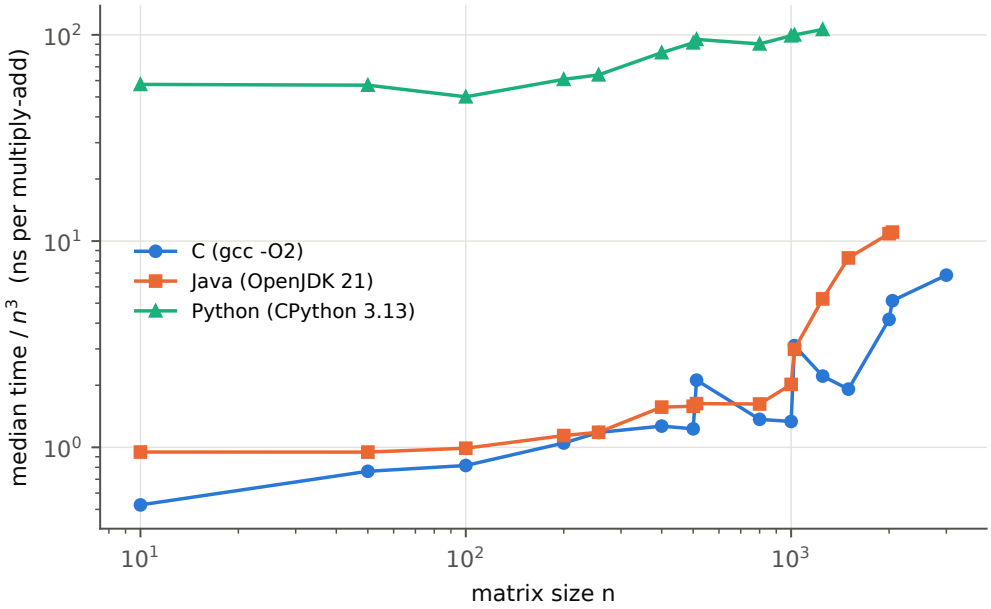



**Figure 2:** Median time divided by $n^3$ (nanoseconds per multiply–add). A purely cubic algorithm with constant cost per operation would give horizontal lines.

**Relative speed.** For $50 \le n \le 1000$ Python is 54–75× slower than C and 49–60× slower than Java; over the whole range where both exist the Python/C ratio is 32× (at $n = 1024$, where C is slow, see below) to 109× ($n = 10$). Java is within 0.77–1.51× of C for $50 \le n \le 1024$ (1.80× at $n = 10$), but 2.4× ($n = 1250$), 4.3× ($n = 1500$), 2.6× ($n = 2000$) and 2.2× ($n = 2048$) slower at larger sizes (`results/summary_ratios.csv`).

## 4.3 Memory

Table 5 and Figure 3 give the peak resident memory (`VmHWM`). Memory was never the limiting factor: the largest run (C, $n = 3000$) peaked at 207.8 MiB, versus the 4 GiB budget.

| $n$ | C | Java | Python |
|---|---|---|---|
| 10 | 1.8 | 45.7 | 9.8 |
| 50 | 1.8 | 43.2 | 10.0 |
| 100 | 2.0 | 43.8 | 10.9 |
| 200 | 2.7 | 44.5 | 14.3 |
| 256 | 3.3 | 45.0 | 17.4 |
| 400 | 5.4 | 47.2 | 28.3 |
| 500 | 7.5 | 49.2 | 38.7 |
| 512 | 7.8 | 49.7 | 40.1 |
| 800 | 16.4 | 57.6 | 83.6 |
| 1000 | 24.7 | 66.5 | 125.0 |
| 1024 | 25.8 | 68.0 | 130.6 |
| 1250 | 37.5 | 79.3 | 189.6 |
| 1500 | 53.3 | 94.9 | – |
| 2000 | 93.3 | 174.1 | – |
| 2048 | 97.8 | 174.3 | – |
| 3000 | 207.8 | – | – |

**Table 5:** Peak resident memory (MiB) of the process (`VmHWM`); "–" where the run did not complete.

| Impl. | Baseline (MiB, $n$=10) | Measured bytes per $n^2$ | Model bytes per $n^2$ |
|---|---|---|---|
| C | 1.8 | 24.0 | 24 |
| Java | 45.7 | 22.3 | 24 |
| Python | 9.8 | 120.9 | 120 |

**Table 6:** Baseline (smallest run, $n = 10$) and measured growth in bytes per $n^2$ (median over $n \geq 500$) against the model of three matrices (24 B for C/Java, $\approx$ 120 B for Python).

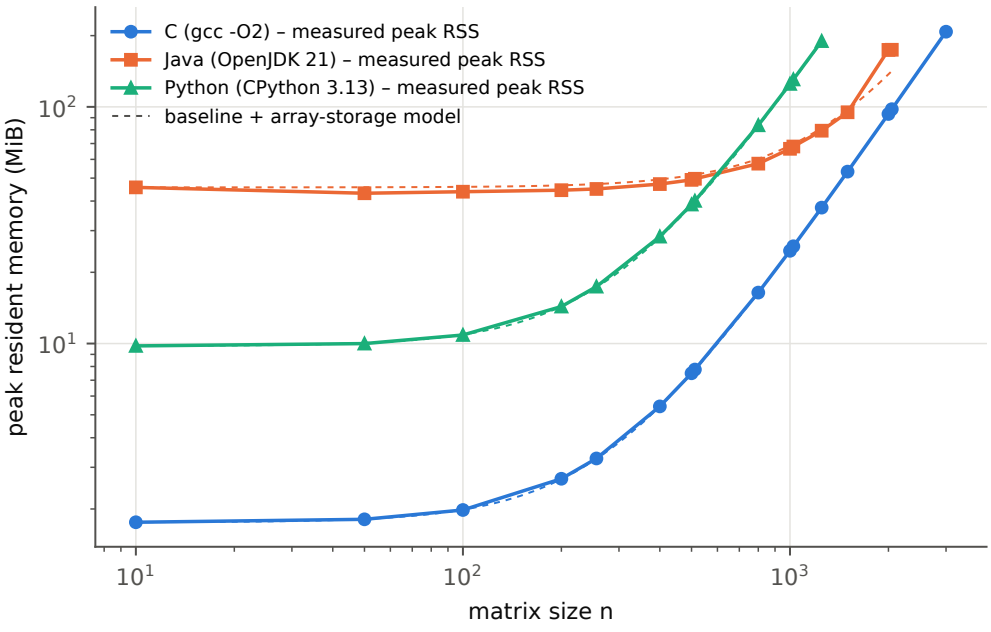



**Figure 3:** Peak resident memory against $n$. Dashed lines: measured baseline of the smallest run plus the array-storage model ($24n^2$ B for C and Java, $120n^2$ B for Python).

C grows by 24.0 bytes per $n^2$ (three 8-byte matrices) on top of a 1.8 MiB baseline. Python grows by 120.9 bytes per $n^2$ (5× more) on top of 9.8 MiB, matching an 8-byte pointer plus a 32-byte float object per element for three matrices. Java has the largest baseline (45.7 MiB, the JVM itself) and grows by $\approx$ 22 bytes per $n^2$ up to $n = 1500$; at $n = 2000$ and 2048 it grows by 32–34 bytes per $n^2$, i.e. about 32 MiB more than the three arrays (not investigated; plausibly heap sizing by the garbage collector). The kernel's

`ru_maxrss` (stored as a cross-check) is useless for small runs (≈ 33 MiB even for C at $n = 10$) because it counts the parent driver's memory at process creation; this is why `VmHWM` was chosen as the primary metric.

## 4.4 Feasible sizes and limits

Table 7 separates the largest size that completed within the declared limits from the largest size attempted. No run failed for memory. All failures are timeouts of a single multiplication at the 300 s limit; their times are *not* known, only that they exceeded 300 s.

| Impl. | Largest feasible $n$ | Largest attempted $n$ | Outcome at largest attempted |
|---|---|---|---|
| C | 3000 | 4096 | timeout (>300 s per sample) |
| Java | 2048 | 3000 | timeout (>300 s per sample) |
| Python | 1250 | 1500 | timeout (>300 s per sample) |

**Table 7:** Largest feasible and largest attempted size per implementation. The largest feasible size of every implementation was measured with only 2 repetitions.

## 4.5 Variability and warm-up

Within a process, the relative IQR is mostly between 0.5 and 20 % (Figure 4, left), with one outlier of 36 % (C, $n = 1250$); the median over all configurations is 4.8 %. The first warm-up sample of Java is up to 1.3–1.6× the steady median for some sizes ($n = 50$, 400, 500) and about 1.0 otherwise (right); with 3 warm-up samples for the sizes below 1 s the measured samples are in steady state. For the long runs (no warm-up) the first measured sample was within 10 % of the median (largest: +10 %, C at $n = 2048$).

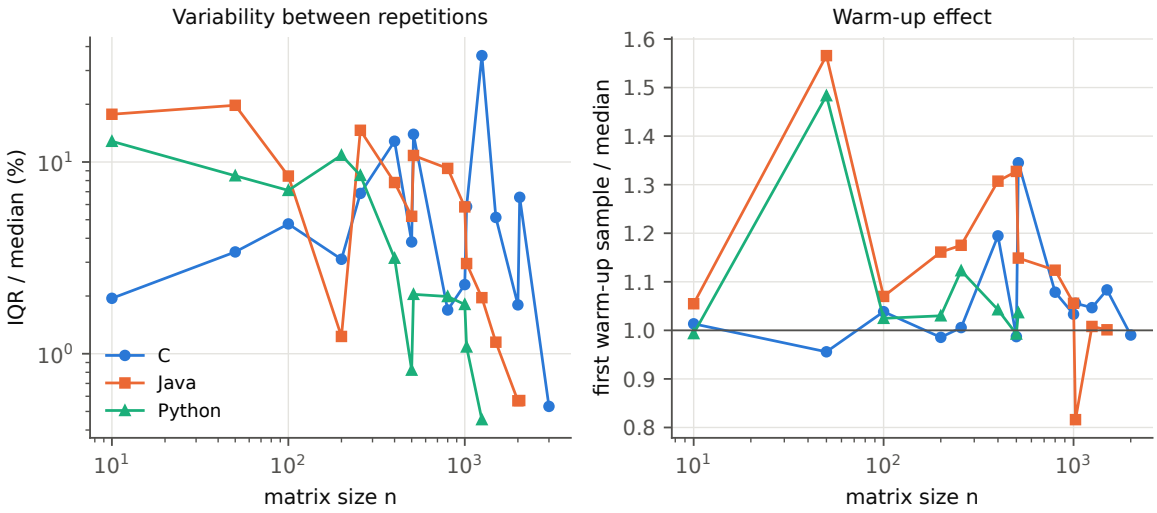



**Figure 4:** Left: relative IQR of the measured samples within one process. Right: first warm-up sample divided by the median of the measured samples (only sizes with warm-up).

## 4.6 Supplementary experiments

**C optimisation level.** Table 8 compares gcc optimisation levels. `-O0` is 2.2–3.7× slower than `-O2`; `-O1`, `-O2`, `-O3` and `-O3 -march=native` are indistinguishable at $n = 512$ and $n = 1000$ (±6 %) and differ by at most 10 % at $n = 1024$, which is within the between-process variability of Section 4.6. All variants produce the same bits as `-O2`. The primary configuration `-O2` is therefore representative of the optimised builds for this kernel; no vectorisation or loop interchange benefit was observed.

| Flags | $n$=512 | $n$=1000 | $n$=1024 | same bits as -O2 |
|---|---|---|---|---|
| gcc -O0 | 638 ms | 5.17 s | 6.63 s | yes |
| gcc -O1 | 299 ms | 1.41 s | 2.75 s | yes |
| gcc -O2 | 292 ms | 1.41 s | 2.7 s | yes |
| gcc -O3 | 267 ms | 1.43 s | 2.97 s | yes |
| gcc -O3 -march=native | 278 ms | 1.39 s | 2.91 s | yes |

**Table 8:** C compiler options: median time per multiplication (10 repetitions at $n = 512$, 5 otherwise).

**Sizes around powers of two.** Table 9 and Figure 5 show the time per multiply–add just below, at and above $2^9$ and $2^{10}$. In C the time at $n = 512$ and $n = 1024$ is about 1.8–1.9× that of the average of the neighbouring sizes ($n = 512$: 2.41 vs. 1.32–1.42 ns; $n = 1024$: 3.04 vs. 1.50–1.67 ns). The Java program with a flat array shows the same spikes (1.7–1.8×), whereas the Java program with `double[n][n]` shows none.

| $n$ | C | Java `double[n][n]` | Java flat `double[]` |
|---|---|---|---|
| 496 | 1.417 | 1.631 | 1.411 |
| 504 | 1.313 | 1.652 | 1.347 |
| 512 | 2.410 | 1.576 | 2.325 |
| 520 | 1.316 | 1.652 | 1.420 |
| 1016 | 1.503 | 2.273 | 1.808 |
| 1024 | 3.043 | 1.956 | 3.327 |
| 1032 | 1.666 | 2.336 | 1.900 |

**Table 9:** Median time per multiply–add (ns) around powers of two. Single process per cell.

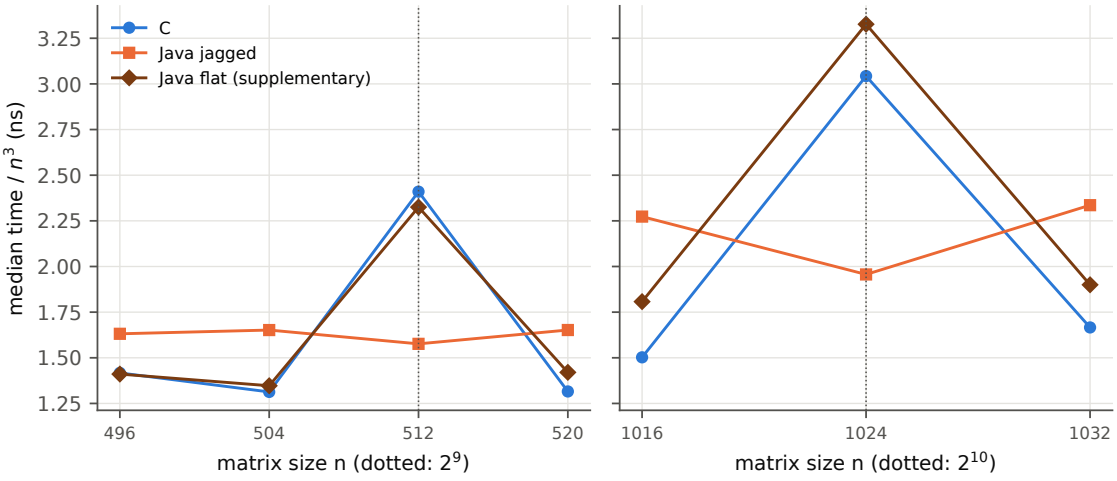



**Figure 5:** Median time per multiply–add around $n = 2^9$ (left) and $n = 2^{10}$ (right).

**Java array layout.** Table 10 and Figure 6 compare jagged and flat storage in Java against C. The flat version is 1.07–1.49× slower than C for $500 \le n \le 2000$, whereas the jagged version is 1.29–4.33× slower; the jagged/flat ratio grows from 1.1 at $n = 500$–1000 to 1.7–2.9 at $n \ge 1250$. The $n = 1250$ C value has an IQR of 36 % and should be read with care.

| $n$ | C | Java jagged | Java flat | jagged/C | flat/C |
|---|---|---|---|---|---|
| 500 | 154 ms | 198 ms | 169 ms | 1.29 | 1.10 |
| 1000 | 1.33 s | 2.02 s | 1.82 s | 1.51 | 1.36 |
| 1250 | 4.33 s | 10.2 s | 5.86 s | 2.37 | 1.36 |
| 1500 | 6.46 s | 28 s | 9.66 s | 4.33 | 1.49 |
| 2000 | 33.4 s | 86.9 s | 35.9 s | 2.60 | 1.07 |

**Table 10:** Median time per multiplication for C, Java with `double[n][n]` (jagged), and Java with a flat `double[]` (supplementary), and ratios.

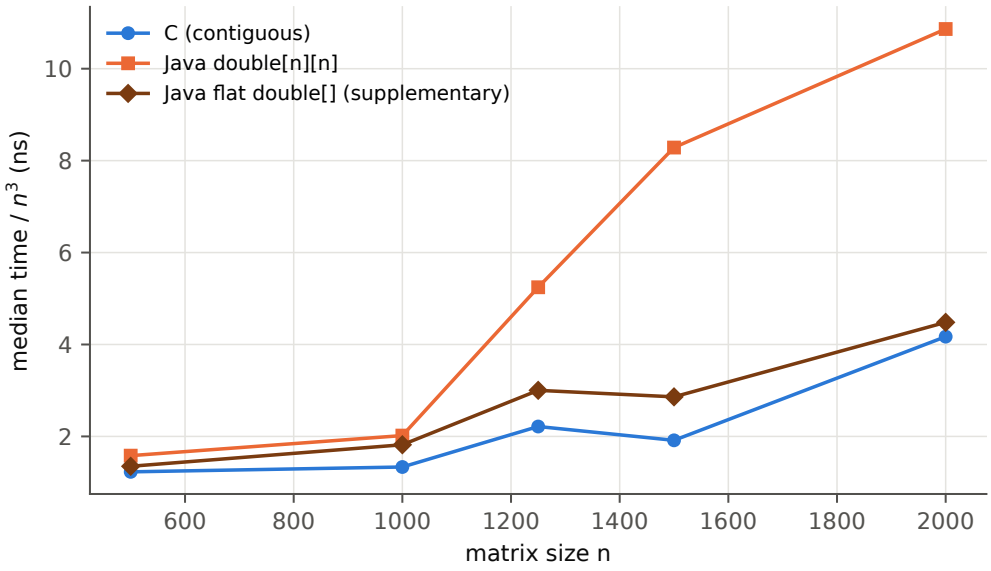



**Figure 6:** Time per multiply–add for C and the two Java layouts.

**Between-process variability.** Table 11 reports the median time at $n = 1024$ in six independent processes. For C the six medians span 13 % of their median (2.96–3.38 s), for Java 40 % (2.10–3.21 s). Within each process the relative IQR is 2–12 % for C and 3–28 % for Java (the min–max range of Java reaches 47 % of the median), so at this size the timing is noisy both within and between processes. A single-process median at $n = 1024$ is therefore uncertain by tens of percent for Java, and differences of that size between C and Java at this $n$ cannot be resolved.

| Independent process | C | Java |
|---|---|---|
| 1 | 3.34 s | 3.21 s |
| 2 | 3.27 s | 2.1 s |
| 3 | 3.06 s | 2.44 s |
| 4 | 2.96 s | 2.75 s |
| 5 | 3.35 s | 2.77 s |
| 6 | 3.38 s | 3.08 s |
| *min* | 2.96 s | 2.1 s |
| *median* | 3.3 s | 2.76 s |
| *max* | 3.38 s | 3.21 s |
| range / median | 13 % | 40 % |

**Table 11:** Median time per multiplication at $n = 1024$ in six independent processes (process 1 is the main run).

# 5 Discussion

## 5.1 Answer to the research question

In this environment, the time of the basic $O(n^3)$ multiplication grows close to cubically for small and medium matrices ($k \approx 3.2$ for $50 \le n \le 400$ in all three languages), but the cost per multiply–add *increases* for larger matrices in the compiled/JIT languages: in C from 1.0–1.4 ns ($200 \le n \le 1000$, excluding $n = 512$) to 6.8 ns ($n = 3000$), in Java from $\approx$ 1.6 ns to 11.1 ns ($n = 2048$); in Python it rises from 50–60 ns to $\approx$ 100 ns. Python is 49–75 times slower than C and Java at $50 \le n \le 1000$. C and Java are comparable for $50 \le n \lesssim 1000$ (ratio 0.77–1.51), after which the `double[n][n]` Java program falls behind (2.2–4.3×). Memory is explained by array storage plus a runtime baseline: 24, $\approx$ 22 (Java, $n \le 1500$) and $\approx$ 121 bytes per $n^2$, with baselines of 1.8, 45.7 and 9.8 MiB for C, Java and Python. Time, not memory, limits the feasible size: $n = 3000$ (C), 2048 (Java), 1250 (Python) under a 300 s-per-multiplication limit.

## 5.2 Hypotheses

- **H1 (cubic growth)**: supported only partially. The exponent is 3.17–3.27 for $n \le 400$, but 3.29 (Python), 3.63 (C) and 4.01 (Java) over the larger windows. The operation count is cubic; the *time* is not, because the cost per operation depends on $n$ (Figure 2).
- **H2 (C $\lesssim$ Java $\ll$ Python)**: Python is 49–75× slower for $50 \le n \le 1000$, consistent with "one to two orders of magnitude". C $\lesssim$ Java holds only for $50 \le n \lesssim 1000$ (within noise); beyond that Java is up to 4.3× slower.
- **H3 (cost per operation rises with $n$)**: supported in all three languages (C 0.5 $\to$ 6.8 ns, Java 0.9 $\to$ 11.1 ns, Python 50 $\to$ 106 ns), with additional spikes at powers of two.
- **H4 (memory model)**: supported for C (24.0 B/$n^2$) and Python (120.9 B/$n^2$); Java matches (22.3) up to $n = 1500$ but needs $\approx$ 32 B/$n^2$ at $n \ge 2000$.

## 5.3 Interpretation, and how strong the evidence is

**Python.** The $\approx$ 50–60 ns per multiply–add for $n \le 200$ is the cost of executing the inner loop in the interpreter, where each iteration performs several bytecode dispatches, two double indexings and the allocation of a float object; this is consistent with the observed constant per-operation cost at small $n$ and the $\approx$ 50–75× gap, but the cost of individual bytecodes was not measured here. The further increase to 82–106 ns at $n \ge 400$ coincides with the working set growing beyond the caches (about 40 bytes per element, Table 5), where every access dereferences a pointer to a float object; Python's increase ($\approx 2\times$) is smaller than in C ($\approx 5\times$) because interpreter overhead is a large constant part of its cost.

**C: powers of two and large $n$.** Two effects are visible. (i) At $n = 512$ and 1024 the matrix row stride is a multiple of 4096 bytes, so the successive elements $b_{kj}$ of a column are mapped to few cache sets; the time per multiply–add jumps by $\approx$ 1.8–1.9× relative to $n\pm$ a few (Figure 5). The effect appears in C and in Java with a flat array, and not in Java with `double[n][n]`, which is consistent with a memory-layout explanation (row addresses of separately allocated Java arrays are not exact multiples of the page size) [2, 3]. (ii) For $n \gtrsim 1250$ the cost per operation grows steadily (Figure 2). The matrix $B$ occupies $8n^2$ bytes (8 MB at $n = 1000$, 32 MB at $n = 2048$, 72 MB at $n = 3000$), which exceeds the L2 cache (2 MiB per core) and approaches or exceeds the L3 cache (33 MiB, shared) of this machine, and a column walk touches a different 4 KiB page at every step for $n \ge 512$, which also stresses the address-translation caches. These are plausible causes, consistent with the trend, but **they were not measured**: no hardware performance counters were available in the virtual machine, so cache misses and TLB misses were not observed.

**Java versus C.** The gap in Figure 1 is not a property of the JIT arithmetic: Java with a flat array is within 1.07–1.49× of C for $500 \le n \le 2000$, whereas the `double[n][n]` version is up to 4.3× slower, and the jagged/flat ratio grows with $n$. A jagged array adds, for every $b[k][j]$, a load of the row reference and a length check on a *separate* object, and rows are separate memory blocks; at large $n$ this plausibly increases cache and TLB pressure. The experiment supports "layout explains most of the gap" but does not identify the mechanism, and the remaining 7–49 % (bounds checks, JIT code quality, or other effects) was not isolated. Warm-up played a minor role: the first warm-up sample was at most 1.6× the steady value and the measured samples were stable (Figure 4).

**C compiler options.** `-O0` is 2.2–3.7× slower; `-O1` to `-O3` are indistinguishable within the noise. Because floating-point reductions cannot be re-associated without `-ffast-math` and the column access of $B$ is strided, the compiler has little to improve; `-march=native` did not help either. This supports using `-O2` as a representative optimised configuration for this particular kernel, nothing more.

### 5.4 Limitations

- **One virtual machine.** All data come from one 2-vCPU KVM guest on a shared host: other tenants, frequency scaling and host scheduling are not observable or controlled, and the processes were not pinned. Absolute times are not transferable to other machines.
- **Between-process variability.** At $n = 1024$ the median of six independent processes varies by 13 % (C) and 40 % (Java) and the within-process spread of Java is also large. Most comparisons of ratios close to 1 at a single size (e.g. Java/C = 0.77 at $n = 512$ or 0.96 at $n = 1024$) are therefore *not* significant. The IQRs in Table 3 describe within-process repetitions only.
- **Few repetitions at large** $n$ (2–3, forced by the budget), a single pair of input matrices per size, and a single run order per size (interleaved by language to reduce, not eliminate, time-of-day effects).
- **Exploratory supplementary experiments.** They were designed after seeing the main results, have one process per cell (except $n = 1024$) and test explanations indirectly; the mechanisms above remain hypotheses.
- **Scope.** Results apply to gcc 13.3 `-O2`, OpenJDK 21 with the serial GC, CPython 3.13 and the algorithm, layouts and inputs described here. They do not support a statement that one language is faster than another in general; other compilers, JVM settings, Python implementations or matrix libraries could change the picture drastically (and are the topic of the next assignments).
- **Memory metric.** RSS is not the same as heap use or allocated storage; the Java excess at $n \geq 2000$ was not investigated.

## 6 Conclusions

1. Under the described conditions, the basic triple-loop multiplication takes $\approx n^{3.2}$ for $n \leq 400$ in C, Java and Python, and the cost per multiply–add rises for larger matrices (C up to 6.8 ns at $n = 3000$, Java 11.1 ns at $n = 2048$, Python $\approx 100$ ns at $n \approx 1000$–$1250$).
2. Python is 49–75× slower than C and Java for $50 \leq n \leq 1000$. C and Java agree within a factor of 1.5 for $50 \leq n \lesssim 1000$; beyond, `double[n][n]` Java is 2.2–4.3× slower than C, and a flat-array Java variant removes most of that gap ($\leq 1.5\times$ of C).
3. C (and flat Java) is $\approx$ 1.7–1.9× slower at $n = 512$ and $n = 1024$ than at neighbouring sizes.
4. Memory follows array storage plus a runtime baseline: 24 B/$n^2$ (C), $\approx 22$ (Java, $n \leq 1500$), $\approx 121$ (Python), with baselines 1.8, 45.7 and 9.8 MiB.
5. With a 300 s per-multiplication limit the largest feasible sizes were 3000 (C), 2048 (Java) and 1250 (Python); the first failures (4096, 3000, 1500) were timeouts, never memory failures.
6. Unresolved: the hardware mechanisms behind the large-$n$ slowdown and the power-of-two spikes (needs performance counters), the origin of Java's extra memory at $n \geq 2000$, and the effect of the machine (needs repetition on other hardware).

The numbers apply to this experiment only; the optimisation techniques that address several of the observed effects (loop order, blocking, sparse formats) are left for Assignment 2.

## 7 Use of generative AI

In line with the course policy, the use of AI is disclosed: the tool was **Claude (Anthropic)**. It was used for planning the experimental design, writing the benchmark driver, the test and analysis scripts, the supplementary experiments and a first draft of this report. The affected parts are `scripts/`, `src/` and `report/`. The author is responsible for understanding the code, validating the measurements, checking the

references and defending the conclusions; generated text is not evidence, and every number in this report was produced by the included code from the raw data in `results/`.

# 8 Reproducibility materials

Source code, build and run instructions, raw measurements, input-generation scripts and analysis scripts are in the repository `https://github.com/hugogzfz/bd-assignment1-matmul` and in `Assignment01_GonzalezFernandez_Materials.zip`. The `README.md` gives the exact commands (`make build inputs test bench cflags analyze report`); the input matrices are not stored but regenerated with `scripts/gen_inputs.py` and verified against the SHA-256 digests in `data/manifest.json`. Raw data: `results/raw_measurements.csv` (one row per sample, warm-up included) and `results/runs.csv` (one row per process); environment: `results/environment.json`; every figure and table is generated by `scripts/analyze.py`. No material from other sources is reused. All numbers in this report come from the run documented in `results/`.

# References


[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., Nov. 2007.

[3] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[4] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[5] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Comput. Surv.*, vol. 23, no. 1, pp. 5–48, Mar. 1991.

[6] A. Georges, D. Buytaert and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, Montreal, Canada, 2007, pp. 57–76.

[7] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC '15)*, Austin, TX, USA, 2015, Art. no. 73.

[8] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[9] Python Software Foundation, "What's new in Python 3.12," Python documentation (built-in `sum()` uses compensated summation for floats; CPython issue gh-100425).